

Fixing SDXL VAE Firefly Artifacts

A Lightweight Traditional CV Approach for Impulse Noise Removal

Image Processing and Visual Effects | Midterm Project Proposal

National Yang Ming Chiao Tung University | Spring 2026

What Are VAE Firefly Artifacts?

The Problem

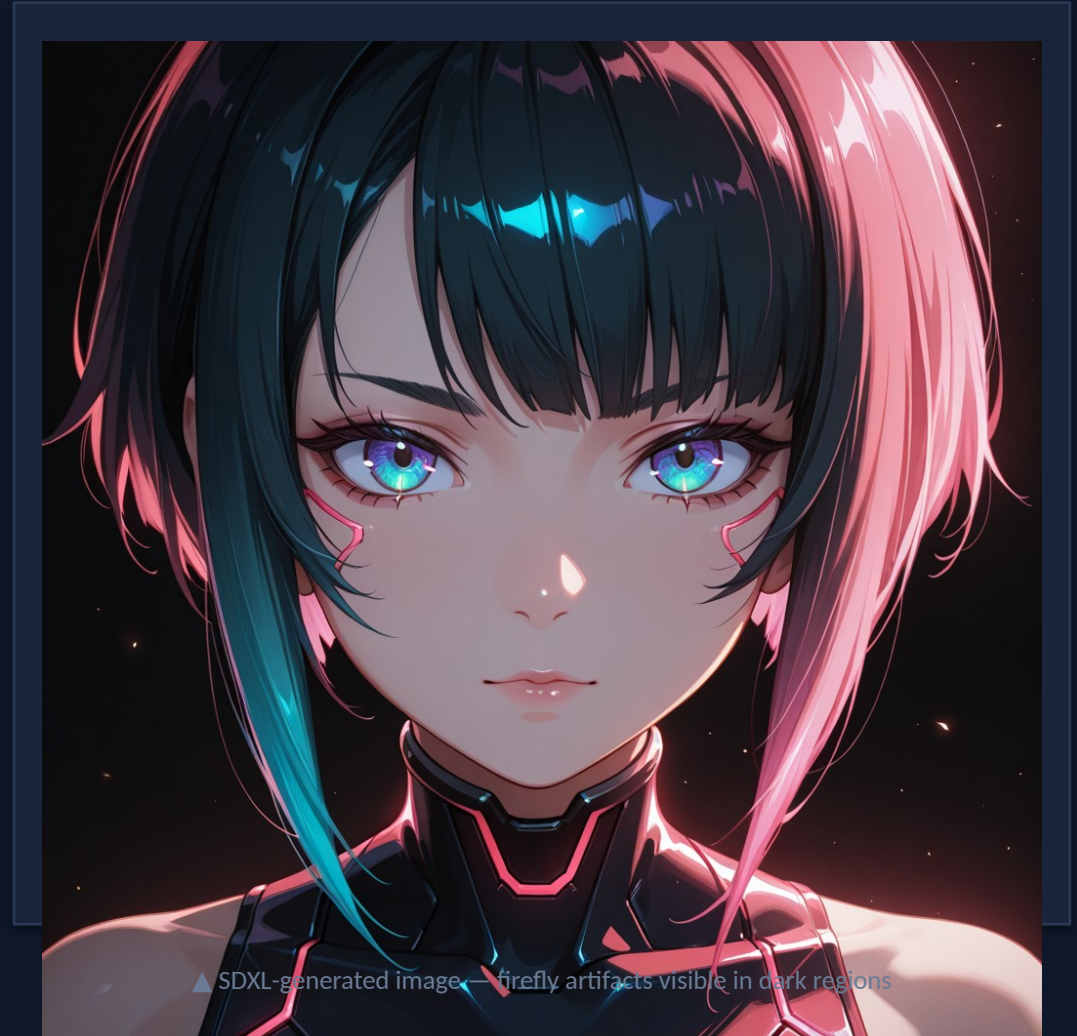
SDXL VAE decoder produces isolated pure-white / pure-color dead pixels (1~5 px) in dark or high-contrast regions

Root Cause

- ▶ VAE 8× spatial compression with only 4 channels → bandwidth limit
- ▶ High CFG Scale pushes latent values out-of-distribution
- ▶ Decoder clips extreme values → impulse noise in pixel space

Current Fix Is Overkill

- ▶ Hires. fix / U-Net re-denoise for < 0.01% of pixels
- ▶ 4K attention = massive VRAM, may OOM on consumer GPU
- ▶ Global re-denoise destroys fine details in non-damaged areas



▲ SDXL-generated image — firefly artifacts visible in dark regions

Key Insight: Why Traditional CV Is the Right Tool

VAE Firefly Noise — Geometric Signature

- ▶ Impulse signal: pixel values jump from ~10 to 255 instantly
- ▶ No Gaussian falloff — hard edges, no anti-aliasing
- ▶ Extremely small area: 1~5 isolated pixels
- ▶ Extreme local variance compared to neighbors
- ▶ Deterministic geometric features — not semantic

Natural Highlights (Stars, Specular) — Smooth

- ▶ Gradual brightness falloff: 250 → 200 → 150 → 80
- ▶ Gaussian edge transition with anti-aliasing
- ▶ Larger area with diffuse boundaries
- ▶ Low local variance — continuous gradient
- ▶ Requires semantic understanding to distinguish

→ These are deterministic geometric features, not semantic ones. Traditional CV (Laplacian + CCA) is the optimal tool.

Image Inpainting — Three Paradigms

We examine three representative inpainting methods and evaluate their suitability for micro-scale VAE artifact repair.

0

RePaint

CVPR 2022

Uses pretrained DDPM as generative prior; conditions by sampling unmasked regions during reverse diffusion

Too heavy for micro-pixel repair

0

PatchMatch

SIGGRAPH 2009

Randomized nearest-neighbor search across entire image; propagates best patch matches iteratively

Over-engineered for tiny holes

0

Telea FMM

JGT 2004

Fast Marching Method propagates pixel info inward from boundary; maintains isophote continuity

Perfect fit for 1~5 px repair

RePaint: Diffusion-Based Inpainting

Methodology

- ▶ Uses a pretrained unconditional DDPM as generative prior
- ▶ No mask-specific training needed — conditions at inference time
- ▶ Reverse diffusion: at each step t , replace unmasked region with noised known pixels
- ▶ Key innovation: "resample" (jump back) to re-harmonize generated content
- ▶ Forward-backward in diffusion time → better semantic coherence

Reverse Diffusion Process

- ▶ ① Start with Gaussian noise x_T
- ▶ ② For each step $t = T \rightarrow 0$:
 - Denoise: $x_{t-1} = \text{DDPM}(x_t)$
 - Replace known region: $x_{t-1}^{\text{known}} = \sqrt{\alpha_t} \cdot x_0 + \text{noise}$
 - Merge: $x_{t-1} = m \odot x_{t-1}^{\text{known}} + (1-m) \odot x_{t-1}^{\text{gen}}$
- ▶ ③ Resample: jump back n steps, repeat
- ▶ ④ Output: harmonized inpainted image

Why NOT Suitable for VAE Fireflies?

- ▶ Must run full reverse diffusion (250~1000 steps) even for 3-pixel mask
- ▶ Requires entire 4K image through U-Net attention → 8~12 GB VRAM
- ▶ Global re-denoise alters non-damaged textures & fine details
- ▶ Latency: seconds to minutes per image — unacceptable for batch workflows
- ▶ Semantic generation is powerful but completely unnecessary for impulse noise

PatchMatch: Randomized Patch-Based Synthesis

Methodology

- ▶ Builds Approximate Nearest-Neighbor Field (NNF) across image
- ▶ Core loop: Random Init → Propagation → Random Search
- ▶ Minimizes Sum-of-Squared-Differences (SSD) for patch similarity
- ▶ Multi-scale pyramid: coarse-to-fine iterative refinement
- ▶ Adobe Photoshop "Content-Aware Fill" is built on this algorithm

Three-Phase NNF Algorithm

- ▶ Phase 1: Random Initialization
 - ▶ → Assign random patch offsets to every pixel
- ▶ Phase 2: Propagation
 - ▶ → Good matches spread to adjacent pixels (spatial coherence)
- ▶ Phase 3: Random Search
 - ▶ → Exponentially shrinking windows test random candidates
 - ▶ Iterate phases 2-3 until convergence (~5 iterations)

Strengths

- ▶ Excellent for large object removal (people, logos)
- ▶ Preserves complex textures (grass, bricks, waves)
- ▶ 20-100× faster than previous NNF methods

Why NOT Suitable for VAE Fireflies?

- ▶ Full-image NNF search for 3-pixel hole = massive waste
- ▶ May copy unrelated distant textures → visual artifacts
- ▶ Non-deterministic: random init → slightly different results each run
- ▶ Designed for macro-scale editing, not micro-scale repair

Telea FMM: Fast Marching Boundary Diffusion

Methodology

- ▶ Based on PDE (Partial Differential Equation) & Level-Set theory
- ▶ Treats inpainting as fluid diffusion from boundary inward
- ▶ Fast Marching Method solves the Eikonal equation: $|\nabla T| = 1$
- ▶ Processes boundary pixels in order of distance to known region
- ▶ Weighted average of neighbors preserves isophote continuity

Core Mathematical Model

- ▶ For point p on boundary $\partial\Omega$:
- ▶ $I_q(p) = I(q) + \nabla I(q) \cdot (p - q)$
- ▶ → First-order approximation from known neighbor q
- ▶ Weighted sum over neighborhood $B_\epsilon(p)$:
- ▶ $I(p) = \sum w(p,q) \cdot [I(q) + \nabla I(q) \cdot (p-q)] / \sum w(p,q)$
- ▶ Weight function $w(p,q)$ ensures:
 - Closer pixels contribute more
 - Gradient direction is preserved

Why PERFECT for VAE Firefly Repair?

- ▶ Local boundary propagation → only processes pixels near the hole, no global search
- ▶ Millisecond-level speed for 1~5 pixel holes (under 3 sec for 800×600 full inpaint)
- ▶ Isophote preservation → smooth gradient transitions, invisible repairs
- ▶ Fully deterministic — same input always produces same output
- ▶ Zero VRAM usage — runs entirely on CPU with OpenCV `cv2.inpaint()`
- ▶ Simple to implement: OpenCV one-liner with `inpaintRadius ≈ 3`

FMM Inpainting: Step-by-Step Process

Step 1

Initialize

Classify all pixels:

- KNOWN: outside mask ($T=0$)
- BAND: on mask boundary
- INSIDE: within mask ($T=\infty$)



Step 2

Select Closest

Pick the BAND pixel p with smallest distance T to the original boundary $\partial\Omega_i$. Mark it as KNOWN.



Step 3

Inpaint Pixel

Compute $I(p)$ using weighted average of all KNOWN neighbors in $B_\epsilon(p)$. Weight considers distance, direction, & gradient.



Step 4

Advance Band

Update neighbors of p :

- INSIDE neighbors → BAND
- Recalculate their T values
- Repeat from Step 2

Key Properties

- ▶ Narrow Band structure guarantees pixels are always filled closest-first
- ▶ Isophote lines (equal brightness contours) remain smooth and continuous
- ▶ Complexity: $O(N \log N)$ where N = number of mask pixels — negligible for micro-holes
- ▶ Implementation in OpenCV: `cv2.inpaint(img, mask, radius=3, flags=cv2.INPAINT_TELEA)`

Head-to-Head: Which Method Fits Our Problem?

Criterion	RePaint (CVPR 2022)	PatchMatch (SIGGRAPH 2009)	Telea FMM (JGT 2004)
Mechanism	Reverse diffusion + resample	Randomized NNF patch search	PDE boundary diffusion
Best For	Semantic inpainting large masks	Object removal texture synthesis	Micro-holes smooth gradients
Speed (3px hole)	Seconds ~ Minutes	100s of ms	< 1 ms
VRAM Usage	8~12 GB (U-Net)	0 (CPU)	0 (CPU)
Determinism	Stochastic	Non-deterministic	Fully deterministic
Micro-hole Quality	Overkill, may alter surrounding detail	May introduce alien textures	Invisible repair perfect blending

Verdict: Telea FMM is the optimal choice — zero VRAM, sub-millisecond, perfect micro-hole blending.

Proposed System Architecture

A three-stage pipeline: Detect → Visualize → Repair — all without deep learning.

Stage 1

Detection & Localization

- ▶ Laplacian filter → energy map (captures all sharp edges)
- ▶ Binarize energy map with adaptive threshold
- ▶ Connected Components Analysis (CCA) → filter by area < max_noise_size
- ▶ Neighborhood color variance check → distinguish noise vs. real detail

Binary mask of confirmed noise pixels



Stage 2

Visualization & Tuning

- ▶ Preview mode: red overlay mask on original image
- ▶ User-adjustable parameters: gradient_sensitivity, max_noise_size
- ▶ Real-time feedback loop for threshold calibration
- ▶ Boolean debug switch: preview_mask ON/OFF

Fine-tuned, validated noise mask



Stage 3

Inpainting (Telea FMM)

- ▶ cv2.inpaint(img, mask, radius=3, cv2.INPAINT_TELEA)
- ▶ Boundary diffusion fills micro-holes in < 1 ms
- ▶ Zero VRAM — runs entirely on CPU via OpenCV
- ▶ Deterministic output — perfect reproducibility

Clean image, artifacts removed

Detection Pipeline: From Pixels to Mask

Laplacian Filter

Apply Laplacian kernel to generate energy map.
High energy = sharp edges and impulse noise.

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$$



Adaptive Binarize

Threshold the energy map based on resolution-aware dynamic sensitivity param.
Output: binary candidates.

$$B(x,y) = E(x,y) > \theta$$



CCA Area Filter

Connected Components Analysis extracts blobs.
Filter: area < max_size keeps only micro-spots.

`cv2.connectedComponentsWithStats`



Variance Check

Compare center vs. 5x5 neighborhood Euclidean color distance. High gap = noise, low = natural.

$$d(c, N) = ||c - \mu_N||_2 > \tau$$

Design Advantages

- ▶ Multi-layer filtering minimizes both false positives (stars/highlights) and false negatives (colored artifacts)
- ▶ Resolution-aware thresholds: parameters auto-scale with image dimensions
- ▶ GPU pre-processing (PyTorch) + CPU geometric analysis (OpenCV) = hybrid architecture for max performance

Summary & Next Steps

Key Takeaways

- ▶ SDXL VAE fireflies are impulse noise — a geometric, not semantic, problem
- ▶ RePaint (diffusion): powerful but wasteful for micro-pixel repair
- ▶ PatchMatch (NNF): excellent for large holes, overkill for tiny ones
- ▶ Telea FMM (PDE): mathematically optimal for 1~5 px boundary diffusion
- ▶ Our solution: Laplacian + CCA detection → Telea FMM repair
- ▶ Zero VRAM, sub-millisecond, fully deterministic

Implementation Roadmap

- ▶ Phase 1: Core detection module (Laplacian + CCA pipeline)
- ▶ Phase 2: Telea FMM integration with auto-radius calculation
- ▶ Phase 3: Visual debugging mode (mask preview overlay)
- ▶ Phase 4: Parameter auto-calibration by resolution
- ▶ Phase 5: ComfyUI custom node packaging
- ▶ Phase 6: Benchmark testing (4K, batch, edge cases)

"Use the right tool for the right problem — not the most popular one."